

Pandas Tutorial

Pandas (stands for Python Data Analysis) is an open-source software library designed for data manipulation and analysis.

- Built on top of NumPy, efficiently manages large datasets, offering tools for data cleaning, transformation and analysis.
- Seamlessly integrates with other Python libraries like NumPy, Matplotlib and scikit-learn.
- Revolves around two primary Data structures: Series (1D) and DataFrame (2D)
- Provides methods like `.dropna()` and `.fillna()` to handle missing values seamlessly

Important Facts to Know :

- **DataFrames:** It is a two-dimensional data structure constructed with rows and columns, which is more similar to Excel spreadsheet.
- **pandas:** This name is derived for the term "panel data" which is econometrics terms of data sets.

What is Pandas Used for?

- Reading and writing data from various file formats like CSV, Excel and SQL databases.
- Cleaning and preparing data (handling missing values, filtering, removing duplicates).
- Merging, joining, and reshaping datasets.
- Performing statistical analysis and descriptive statistics.
- Visualizing data quickly.

1. Pandas Series

A Pandas Series is one-dimensional labeled array capable of holding data of any type (integer, string, float, Python objects etc.). The axis labels are collectively called **indexes**. Series is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file.

```
import pandas as pd
```

```
import numpy as np
```

```
s = pd.Series()
```

```
print("Pandas Series: ", s)
```

```
data = np.array(['g', 'e', 'e', 'k', 's'])
```

```
s = pd.Series(data)
```

```
print("Pandas Series:\n", s)
```

```
Pandas Series: Series([], dtype: object)
Pandas Series:
0    g
1    e
2    e
3    k
4    s
dtype: object
```

2. Pandas DataFrame

Pandas DataFrame is a two-dimensional data structure with labeled axes (rows and columns). It is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file. It can be created from lists, dictionaries, a list of dictionaries etc.

```
import pandas as pd
```

```
df = pd.DataFrame()
```

```
print(df)
```

```
lst = ['Geeks', 'For', 'Geeks', 'is', 'portal', 'for', 'Geeks']
```

```
df = pd.DataFrame(lst)
```

```
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
0
0    Geeks
1      For
2    Geeks
3       is
4  portal
5      for
6    Geeks
```

Operations in Pandas

Pandas provides essential operations for working with structured data efficiently. The sections below introduce the most commonly used functionalities with short explanations and simple examples.

1. Loading Data: This operation reads data from files such as CSV, Excel or JSON into a DataFrame.

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
```

```
print(df.head())
```

	name	age	city	category	sales	a	b
0	John	25	Delhi	A	200	5	10
1	Riya	22	Mumbai	B	120	2	3
2	Aman	30	Pune	A	250	7	8
3	Neha	27	Kolkata	C	300	4	6
4	Arjun	35	Jaipur	B	200	1	9

2. Viewing and Exploring Data: After loading data, it is important to understand its structure and content. This methods allow you to inspect rows, summary statistics and metadata.

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   name        5 non-null     object
1   age         5 non-null     int64
2   city        5 non-null     object
3   category    5 non-null     object
4   sales       5 non-null     int64
5   a           5 non-null     int64
6   b           5 non-null     int64
dtypes: int64(4), object(3)
memory usage: 412.0+ bytes
None
```

3. Handling Missing Data: Datasets often contain empty or missing values. Pandas provides functions to detect, remove or replace these values.

```
print(df.isnull().sum())
```

```
df = df.fillna(0)
```

```

name      0
age       0
city      0
category  0
sales     0
a         0
b         0
dtype: int64

```

4. Selecting and Filtering Data: This operation retrieves specific columns, rows or records that match a condition. It allows precise extraction of required information.

```
ages = df[df['age'] > 25]
```

```
print(ages)
```

	name	age	city	category	sales	a	b
2	Aman	30	Pune	A	250	7	8
3	Neha	27	Kolkata	C	300	4	6
4	Arjun	35	Jaipur	B	200	1	9

5. Adding and Removing Columns: You can create new columns based on existing ones or delete unwanted columns from the DataFrame.

```
df['total'] = df['a'] + df['b']
```

```
print(df.head())
```

	name	age	city	category	sales	a	b	total
0	John	25	Delhi	A	200	5	10	15
1	Riya	22	Mumbai	B	120	2	3	5
2	Aman	30	Pune	A	250	7	8	15
3	Neha	27	Kolkata	C	300	4	6	10
4	Arjun	35	Jaipur	B	200	1	9	10

6. Grouping Data (GroupBy): Grouping allows you to organize data into categories and compute values for each group for example, sums, counts or averages.

```
res = df.groupby('category')['sales'].sum()
```

```
print(res)
```

```

category
A      450
B      320
C      300
Name: sales, dtype: int64

```